```
        Result msg;
        if (msgrcv(qid, &msg, sizeof(msg) - sizeof(long), 1, 0) == -1) {
            perror("msgrcv"); return 1;
        }
        printf("Child %ld: sum of 10 numbers = %d\n", (long)msg.pid, msg.sum);
        received += msg.sum;
    }
    for (int i = 0; i < CHILDREN; i++) wait(NULL);
    msgctl(qid, IPC_RMID, NULL);
    printf("Expected total: %d; received total: %d\n", expected, received);
    return expected == received ? 0 : 1;
}
```

**shell | Compile and run the four-child IPC challenge**

```
gcc -std=c11 -Wall -Wextra pipe_queue_four.c -o pipe_queue_four
./pipe_queue_four
# Four lines: Child <PID>: sum of 10 numbers = ...
# Final line: Expected total: X; received total: X
```

**TWO SUBTLE CORRECTNESS ISSUES** Messages may arrive in different orders; do not rely on PID order. Pipes contain bytes, not typed integers or message boundaries. This example preloads 40 integers and each child reads a 10-integer chunk; for production-grade communication with multiple readers, use explicit framing/synchronization or separate pipes per child. Shared memory remains equally important despite this exercise specifically choosing a message queue.

### Self-check before moving on

1. Why must the message structure begin with long mtype?
2. What happens when a receiver asks for type 7 but only type 5 is present?
3. Which call removes a shared-memory segment?

# 5 | Lab 4: file descriptors, unnamed pipes and redirection

**Highest-value programming chapter: Fall 2023, Spring 2024 and Fall 2024 all test pipe/redirect combinations.**

## 5.1 Understand descriptors before writing code

| Descriptor | Name | What goes here normally |
|---|---|---|
| 0 | STDIN_FILENO | Keyboard or input stream |
| 1 | STDOUT_FILENO | Terminal output |
| 2 | STDERR_FILENO | Terminal errors |

pipe(fd) creates a read end fd[0] and write end fd[1]. dup2(oldfd, newfd) makes newfd refer to the same open file description as oldfd, replacing whatever newfd pointed to before. Close the originals after dup2 when they are no longer needed.

**C pattern | A tiny reusable pipe template**

```
int pipefd[2];
pipe(pipefd);          // pipefd[0] read, pipefd[1] write
dup2(pipefd[1], 1);  // Redirect stdout into pipe
close(pipefd[0]);
close(pipefd[1]);
```

**IMPORTANT** Closing unused pipe ends is essential. If any process still holds a write end open, the reader may never see EOF, so commands such as wc or sort can hang indefinitely.

## 5.2 Original Lab 4: redirect stdout to a file

**redirect.c | Open my.file, dup2 onto stdout, write OK**

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int fd = open("my.file", O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd == -1) { perror("open"); return 1; }
    if (dup2(fd, STDOUT_FILENO) == -1) { perror("dup2"); close(fd); return 1; }
    close(fd);  // stdout now refers to the opened file
    if (write(STDOUT_FILENO, "OK\n", 3) != 3) { perror("write"); return 1; }
    return 0;
}
```

**shell | O_APPEND makes successive runs add new lines**

```
gcc -Wall -Wextra redirect.c -o redirect
./redirect
cat my.file
# Repeating ./redirect appends another OK line.
```

open() returns a descriptor; dup2(fd, 1) changes stdout; close(fd) frees the extra descriptor; write(1,...) now writes to my.file. O_CREAT creates the file if needed. O_TRUNC clears an existing file; O_APPEND adds to it.

## 5.3 Original Lab 4: one pipe, two commands

**simpleredirect.c | Equivalent to ls -l | wc; one fork and one exec per process**

```c
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {                  // Child: ls -l
        if (dup2(fd[1], STDOUT_FILENO) == -1) return 1;
        close(fd[0]); close(fd[1]);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp ls"); return 1;
    }
    if (dup2(fd[0], STDIN_FILENO) == -1) return 1; // Parent: wc
    close(fd[0]); close(fd[1]);
    execlp("wc", "wc", (char *)NULL); // Replaces the parent process
    perror("execlp wc");
    return 1;
}
```

**shell | Compare the final counts; actual values depend on directory**

```
gcc -Wall -Wextra simpleredirect.c -o simpleredirect
./simpleredirect
ls -l | wc
```

How it works: pipe is created BEFORE fork. The child connects stdout to fd[1] and execs ls -l. The parent connects stdin to fd[0] and execs wc. Notice that the parent itself becomes wc, so this exact teaching pattern does not call waitpid() afterwards.

## 5.4 Solved Spring 2024 task: five integers, sum and output file

**Spring 2024 | 25 marks | Pipe + fork + sum + redirect to sum.txt**

Question idea: one process writes 10, 15, 20, 25 and 30 into a pipe; a child reads and sums them; its standard output is redirected to a file called sum.txt. An expected result is 100.

1. Create the pipe before fork() so both processes inherit its descriptors.
2. The parent closes the read end, sends the five values, then closes its write end.
3. The child closes the write end, reads five values from the pipe, and computes the sum.
4. The child opens sum.txt and uses dup2(out, STDOUT_FILENO) before printf.
5. The parent waits until the child is finished; verify the file with cat sum.txt.

**sum_pipe.c | A full, tested solution using stdio parsing on the pipe**

```c
#include <fcntl.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
int main(void) {
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {                         // Child reads and calculates
        close(p[1]);
        FILE *in = fdopen(p[0], "r");
        if (!in) { perror("fdopen"); return 1; }
        int value, sum = 0;
        for (int i = 0; i < 5; i++) {
            if (fscanf(in, "%d", &value) != 1) {
                fprintf(stderr, "Expected five integers\n"); return 1;
            }
            sum += value;
        }
        fclose(in);
        int out = open("sum.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (out == -1) { perror("open"); return 1; }
        if (dup2(out, STDOUT_FILENO) == -1) { perror("dup2"); return 1; }
        close(out);
        printf("10+15+20+25+30 = %d\n", sum);
        return 0;
    }
    close(p[0]);                            // Parent writes numbers
    const char data[] = "10 15 20 25 30\n";
    if (write(p[1], data, sizeof(data) - 1) != sizeof(data) - 1) {
        perror("write"); return 1;
    }
    close(p[1]);
    if (waitpid(pid, NULL, 0) == -1) { perror("waitpid"); return 1; }
    return 0;
}
```

**shell | Compilation and verification**

```shell
gcc -Wall -Wextra -std=gnu11 sum_pipe.c -o sum_pipe
./sum_pipe
cat sum.txt
# 10+15+20+25+30 = 100
```

Why fdopen? It wraps the pipe read descriptor as a FILE* so fscanf can safely parse whitespace-separated integers from a byte stream. The system call pipe sends bytes, not typed integers. After parsing, the child writes to the redirected stdout.

> **TECHNICAL DETAIL** Do not assume read() always returns exactly the number of bytes you requested. For a general binary protocol, loop until all required bytes arrive or handle EOF. Parsing a text stream with fscanf is straightforward for this exam scenario.

## 5.5 Solved Fall 2023 task: three-stage command pipeline

### Fall 2023 | 25 marks | Implement ls | grep c | sort in C

**text | One pipe per connection: three commands require TWO pipes**

```
ls --stdout--> pipe A --stdin--> grep c --stdout--> pipe B --stdin--> sort
```

The provided paper is a line-ordering problem. The solution below uses three children and a waiting parent rather than reproducing the paper's exact two-child/parent-exec layout; it implements the same shell pipeline correctly and is easier to debug.

1. Call pipe(a) and pipe(b) before forking.
2. First child: dup2(a[1], stdout); execute ls.
3. Second child: dup2(a[0], stdin), dup2(b[1], stdout); execute grep c.
4. Third child: dup2(b[0], stdin); execute sort.
5. Every process closes every unused pipe descriptor; parent waits for children.

**pipeline3.c | Three processes for ls, grep and sort; two pipes**

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
int main(void) {
    int a[2], b[2];
    if (pipe(a) == -1 || pipe(b) == -1) { perror("pipe"); return 1; }
    pid_t p1 = fork();
    if (p1 == -1) { perror("fork ls"); return 1; }
    if (p1 == 0) {                  // ls -> first pipe
        dup2(a[1], STDOUT_FILENO);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("ls", "ls", (char *)NULL);
        perror("ls"); return 1;
    }
    pid_t p2 = fork();
    if (p2 == -1) { perror("fork grep"); return 1; }
    if (p2 == 0) {                  // first pipe -> grep -> second pipe
        dup2(a[0], STDIN_FILENO);
        dup2(b[1], STDOUT_FILENO);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("grep", "grep", "c", (char *)NULL);
        perror("grep"); return 1;
    }
    pid_t p3 = fork();
    if (p3 == -1) { perror("fork sort"); return 1; }
    if (p3 == 0) {                  // second pipe -> sort -> terminal
        dup2(b[0], STDIN_FILENO);
        close(a[0]); close(a[1]); close(b[0]); close(b[1]);
        execlp("sort", "sort", (char *)NULL);
        perror("sort"); return 1;
    }
    close(a[0]); close(a[1]); close(b[0]); close(b[1]);
    waitpid(p1, NULL, 0); waitpid(p2, NULL, 0); waitpid(p3, NULL, 0);
    return 0;
}
```

**shell | Compare with the real shell pipeline**

```
gcc -Wall -Wextra pipeline3.c -o pipeline3
./pipeline3
ls | grep c | sort
```

For the paper's rearrange-lines format, start with declarations/includes, create both pipes, fork for the first and second command, assign dup2 endpoints in each branch, close unused ends, then exec. The crucial points are the direction of each descriptor and the order of fork, dup2 and exec.

## 5.6 Solved Fall 2024 task: cat + sort + output redirection

**Fall 2024 | 5 marks | Implement cat /etc/hosts | sort > output.txt**

This is another combination problem. The command on the right must receive bytes from the pipe via stdin AND send its output to output.txt via stdout.

**catsort.c | Correct C solution with two children, one pipe and output redirection**

```c
#include <fcntl.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
int main(void) {
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); return 1; }
    pid_t a = fork();
    if (a == -1) { perror("fork"); return 1; }
    if (a == 0) { // cat /etc/hosts -> pipe
        dup2(p[1], STDOUT_FILENO);
        close(p[0]); close(p[1]);
        execlp("cat", "cat", "/etc/hosts", (char *)NULL);
        perror("cat"); return 1;
    }
    pid_t b = fork();
    if (b == -1) { perror("fork"); return 1; }
    if (b == 0) { // pipe -> sort -> output.txt
        int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (out == -1) { perror("open"); return 1; }
        dup2(p[0], STDIN_FILENO);
        dup2(out, STDOUT_FILENO);
        close(out); close(p[0]); close(p[1]);
        execlp("sort", "sort", (char *)NULL);
        perror("sort"); return 1;
    }
    close(p[0]); close(p[1]);
    waitpid(a, NULL, 0); waitpid(b, NULL, 0);
    return 0;
}
```

**shell | Check the result with diff**

```
gcc -Wall -Wextra catsort.c -o catsort.exe
./catsort.exe
cat output.txt
# Compare: cat /etc/hosts | sort > expected.txt
# diff output.txt expected.txt
```

## 5.7 Pipe problems: a reusable decision table

| Question pattern | Pipes | Process plan |
|---|---|---|
| A \| B | 1 | Child executes A; parent or second child executes B |

| Question pattern | Pipes | Process plan |
|---|---|---|
| A \| B \| C | 2 | One child per command, or 2 children + parent exec |
| A \| B > file | 1 | Connect A to pipe, B stdin to pipe and B stdout to file |
| Parent sends, child sums | 1 | Parent writes; child reads; parent waits |

### Self-check before moving on

1. How many pipes are needed for FOUR commands?
2. Which pipe end becomes stdin of grep in ls | grep c | sort?
3. Why can failing to close a write end freeze sort or wc?

# 6 | Lab 5: named pipes (FIFOs)

### An unnamed pipe is identified by inherited descriptors; a FIFO has a path on the filesystem.

## 6.1 What makes a FIFO different?

mkfifo("demo.fifo", 0600) creates a special filesystem object. Independent processes can open the same path. Opening O_RDONLY can block until a writer appears, and opening O_WRONLY can block until a reader appears. A FIFO is a byte stream, not a permanent text file.

**text | Conceptual data flow; this diagram is not executable C**

```
mkfifo("demo.fifo", 0600)
         |
         +-- parent: open(..., O_RDONLY)  -> read
         +-- child:  open(..., O_WRONLY)  -> write
```

## 6.2 Three-file original lab workflow, rewritten and tested

The manual splits the work into parentchildfifo.c, dofifochild.c and dofifoparent.c. Keeping the three-file layout teaches the linker to combine several .c files into one executable.

**parentchildfifo.c | Creates FIFO, forks, waits and unlinks**

```c
#include <errno.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>
int dofifochild(const char *name);
int dofifoparent(const char *name);
int main(int argc, char **argv) {
    if (argc != 2) { fprintf(stderr, "Usage: %s FIFO\n", argv[0]); return 1; }
    if (mkfifo(argv[1], 0600) == -1 && errno != EEXIST) {
        perror("mkfifo"); return 1;
    }
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) return dofifochild(argv[1]);
    int status = dofifoparent(argv[1]);
    waitpid(pid, NULL, 0);
    unlink(argv[1]);  // Remove filesystem FIFO when finished
    return status;
}
```